Lecture 04: testing and style

Goals

- create circuit test inputs
- use Logisim's testing facilities to verify circuit correctness
- identify characteristics of good style

first two problems ask you to build circuits with particular behaviors

how will you know you've got the correct answer?

in these cases, it's pretty straightforward: both of the circuits have two input pins and therefore four poss
values, and you can just use the Poke tool to set the inputs to each of those four in turn and verify that th

note that it's a really good idea to explicitly write down all the possible inputs and the corresponding desi

it's really easy to forget or skip something

this is where truth tables come very much in handy

what about larger circuits, though?

even our 2-input multiplexer, with eight different input combinations, may feel like it would be cumbersom

fortunately, we've already enumerated exactly the behavior we want in a truth table:

```
 A | B | sel | result
--------------------
 0   0    0      0
 0   0    1      0
 0   1    0      0
 0   1    1      1
 1   0    0      1
 1   0    1      0
 1   1    0      1
 1   1    1      1
```

it would be really nice if we could give this truth table to Logisim and have it test the circuit for us…

and we can!

and in fact the format of the truth table that Logisim wants is pretty much exactly the same as the truth ta

the only difference is that it doesn't want any of the dividing lines

it must be a plain text file (so write it using your text editor, not Word)

the first line must be a list of input and output labels, separated by spaces

each line after that is a single test

since the 2-input mux has 8 different inputs, this table has 8 rows (plus the column headers)

here is the exact contents of the file we want:

```
A   B   sel   result
0   0    0      0
0   0    1      0
```

```
0   1    0      0
0   1    1      1
1   0    0      1
1   0    1      0
1   1    0      1
1   1    1      1
```

to feed it to Logisim, open the circuit file, go to the "Simulate" menu and select "Test Vector…"

in the bottom left, click "Load Vector" and navigate to the text file containing the table above

the tests will automatically run when the file is loaded

if you change the circuit, you have to click "Run" to run the tests again

if there's a difference between the desired output and the actual output, the incorrect output will be highli
show the actual output, not the desired output)

side note: to add the little negation circle to gate inputs, select the gate with the Arrow tool, go to the Pro
"Negate N" property to "Yes"

there is *not* a similar feature to add the little negation circle to the output: for that, you have to select a dif
XNOR)

what about the 4-input mux I showed you at the end of class on Friday?

we can certainly do the same thing

except I'm guessing the idea of writing out the truth table for a 6-input circuit is not terribly exciting and d
learning exercise

I agree

so I wrote a program to produce a table that enumerates all possible inputs

and all we have to do is add the column headers and the output value(s)

it's on `weathertop`:

```
$ 202 truth-table
Usage: 202 truth-table number-of-inputs
```

```
$ 202 truth-table 3
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

now you might be asking yourself: that's all well and good but how do I get it into a file?

Linux has a handy trick here: while normally programs print to the terminal window, you can ask Linux to
sent to a file

you do this by following your command with the greater-than sign and the name of the file you want it to

beware, though, because the file's contents will be replaced!

```
$ 202 truth-table 3 > truth-table.txt
```

the `cat` program will output the contents of a file to the terminal, and we can use that to verify it worked:

```
$ cat truth-table.txt
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1
```

now I can copy the `truth-table.txt` file off `weathertop`, add the column headers, add the output column(test my circuit

you don't have to write tests for the first couple problems, but you will have to do so for later ones (such a stated)

this is one way you can have confidence in the correctness of your submissions

new topic: style

here's the inverter circuit from last week: inverter.circ

and here is a functionally identical circuit: inverter_ugly.circ

even without looking at the file names, which one is better?

most people would probably pick the circuit we created last week

but why?

what is it about the second circuit that makes it so clearly inferior?

it's got wires that needlessly cross

it's got superfluous wires that lead to nowhere

components that fulfill complementary functionality (eg, the power and ground inputs, the pair of transisto differently to each other

the input and the output pins are offset

all of these things make it more difficult to understand the behavior of the circuit by looking at it

and therefore require more thinking to achieve that understanding

and thinking is always error-prone

so the clearer we can make our circuits (and, later, our programs), the less thinking is involved in trying t less opportunity for mistakes

another example is the 2-input mux from Friday: mux_2.circ

and an equivalent circuit: mux_2_ugly.circ

this comparison reveals many similar complaints

one thing that I actually think is worth considering retaining about the "ugly" version is the rotated "sel" in that it is different from the "A" and "B" inputs, which could be helpful in understanding how the circuit wor

and that's the entire point of good style (in both circuits and traditional coding): to make it easy to unders

form should reflect function

regularity is good: discrepancies are easier to spot

if different parts of your circuit behave the same way, they should look the same

logic flow should be apparent

related inputs/outputs should be grouped

inputs and outputs should have descriptive labels

if there is symmetry in the concept or functionality, there should be corresponding symmetry in the circuit

extraneous stuff (wires, components, labels) should be removed

abstraction is an especially effective way to improve style

because, as you will recall, abstraction means hiding details

hiding the details of intricate circuitry inside a black box makes the larger circuit more understandable

the trick is to pick the correct chunk of logic to stuff inside the black box

it takes experience to judge what works well here, but if you find yourself placing the same collection of c
again, consider using Logisim's abstraction feature: subcircuits

here's how:

a Logisim file can store multiple, separate circuits

up 'til now, we've only used one, and you can see it listed in the top-left pane: it's labeled "main"

to create a second circuit in the same file, click the green "+" sign just above

you'll be prompted for a name and here descriptive names are best (though you can't use spaces or hyp
underscores to separate words)

now you can design to your heart's content on a new canvas

any logic you place on this canvas will be inside the black box and any input pins and output pins you pla
outputs of the black box

if we paste the logic for a 2-input mux on this canvas, then we can place a black box on the "main" canva
mux behavior

double-click on the name "main" in the circuit list to switch to that view

now, single-click the 2-input mux circuit in the list to place a 2-input mux in the main circuit

it's important to remember, though:

DOUBLE-CLICK to edit a different circuit

SINGLE-CLICK to place an instance of a subcircuit

you can also change the appearance of the subcircuit (but beware: this is a bottomless bit of creativity th
you let it)

in the list of subcircuits, right-click the circuit whose appearance you want to tweak and click "Edit Circuit

the menu turns into drawing tools, which you can play with

for full details, check Help -> User's Guide -> Subcircuits -> Customize the appearance

note that when you place a subcircuit with customized appearance, you need to set an attribute to actua
the component you've placed and change the Appearance attribute to Custom

(after you do so, any further instances of the same component you place should also use the Custom ap

here's the circuit file that has a 4-input mux built from 2-input muxes as subcircuits: mux_4_from_mux_2

note: this functionality of Logisim-Evolution appears (to me) a bit delicate

I strongly recommend that you save often and make backups

Last modified: 09/22/2026 14:21:24